TECHNICAL REPORT & ENGINEERING REFERENCE MANUAL

HYDROAJ: AN OPEN-SOURCE, SIMD-ACCELERATED COMPUTATIONAL FRAMEWORK FOR WATER RESOURCES ENGINEERING

Rigorous Mathematical Formulations, Parameter Derivations, SI Metric Standards, and Vectorized API Implementation

---

Ajay Bhattarai

Civil Engineer | Bachelor's in Civil Engineering (B.E. Civil)
Systems Architect & Framework Author

Personal Portfolio: https://ajaybhattarai-123.github.io/Personal-website/
LinkedIn Profile: https://www.linkedin.com/in/ajaybhattarai360/
Official GitHub Repository: https://github.com/ajaybhattarai-123/hydroaj
Bug Reports & Issue Tracker: https://github.com/ajaybhattarai-123/hydroaj/issues
Official Python Package Index: https://pypi.org/project/hydroaj/

Publication Date: November 2026 | Version 0.1.1
Standard Publication & Distribution Edition

ABSTRACT

This technical report presents the theoretical formulation, architectural design, and operational implementation of HydroAj, an open-source high-throughput computational framework designed for water resources and civil engineering. Traditional civil engineering hydraulic and hydrological analysis software is frequently constrained by proprietary licenses, monolithic desktop architectures, and severe computational bottlenecks when analyzing regional, multi-catchment, or high-resolution temporal datasets. HydroAj addresses these constraints through a two-tiered hybrid software architecture: a native, memory-safe, SIMD-parallelized computational kernel compiled in Rust with Link-Time Optimization (LTO), bridged via zero-copy buffer passing to an ergonomic, vectorized Python API. The framework incorporates 296 rigorously verified

engineering routines across five core disciplines: open-channel hydraulics, pressurized conduit networks, surface water hydrology, hydraulic control structures, groundwater porous-media mechanics, and fluvial sediment dynamics. Every routine enforces strict, transparent SI metric unit standards with built-in IDE signature assistance. The entire library passes 108 automated unit tests and a 112-problem comprehensive practice workbook with zero numerical discrepancy, demonstrating throughput enhancements of one to two orders of magnitude over standard scripting approaches.

1. ARCHITECTURAL DESIGN & COMPUTATIONAL FOUNDATION

Water resources engineering calculations require both mathematical precision and scalable data execution. Modern engineering applications—ranging from high-resolution 2D hydraulic flood modeling to million-node municipal distribution simulations and continuous watershed runoff modeling—frequently process data arrays containing 10^5 to 10^8 elements. Traditional interpreted Python implementations suffer from severe Global Interpreter Lock (GIL) overhead, memory fragmentation, and dynamic type dispatch penalties.

1.1 The Rust SIMD Kernel (_hydroaj_core)

HydroAj delegates computationally intensive numeric operations to an underlying Rust library (_hydroaj_core). Rust provides strict compile-time memory safety, thread safety without garbage collection overhead, and native Single Instruction Multiple Data (SIMD) vectorization. By leveraging the rayon work-stealing parallelism crate and ndarray continuous buffer operations, HydroAj executes vector operations across all available CPU execution cores. The crate is compiled under Rust 2021 edition with opt-level = 3, codegen-units = 1, and fat Link-Time Optimization (LTO), maximizing loop unrolling and instruction pipelining.

1.2 Zero-Copy Memory Model and PyO3 C-API Bridge

Communication between Python and the native Rust computational kernel occurs via PyO3 and the C-compatible contiguous memory buffer protocol. When a NumPy ndarray or Polars Series is passed to a HydroAj function, the memory pointer is validated and passed directly to the Rust engine without copying bytes in memory. Computations are evaluated in-place or written directly to allocated contiguous output arrays, achieving zero memory duplication overhead.

1.3 Strict SI Metric Standards and IDE Signature Assistance

A recurrent source of engineering error in hydraulic calculations stems from dimensional mismatch (e.g., mixing minutes with hours, or millimeters with meters). HydroAj establishes a strict, unambiguous unit convention: all functions operate by default in standard SI metric units (meters, seconds, m^3/s, hours for storm duration, mm/hr for rainfall intensity, Pascals for pressure). Furthermore, every function docstring embeds an immediate signature summary line (Units (Default SI): ...) enabling parameter hints and tooltips to display exact units directly inside the developer's integrated development environment (VS Code, PyCharm, or Jupyter) upon typing.

2. OPEN CHANNEL HYDRAULICS & PIPE DYNAMICS (hydroaj.hydraulics)

The hydraulics module provides comprehensive routines for steady and unsteady open-channel flow, pipe network resistance, and hydraulic jumps. The primary governing equations, parameter definitions, and corresponding HydroAj function signatures are detailed below.

2.1 Governing Equations & Analytical Formulations

Manning's Equation for Open Channel Discharge:

Equation (1): *Q = (1 / n) * A * (R_h)^(2/3) * (S_0)^(1/2)*
Where Q = Discharge [m³/s], n = Manning roughness [s/m^(1/3)], A = Cross-sectional area [m²], R_h = Hydraulic radius A/P [m], S_0 = Longitudinal channel bed slope [m/m (decimal)].

Chezy Formula for Mean Velocity:

Equation (2): *V = C * sqrt(R_h * S_0)*
Where V = Mean flow velocity [m/s], C = Chezy resistance coefficient [m^(1/2)/s].

Froude Number and Hydraulic Jump Sequent Depth (Belanger Equation):

Equations (3-5): *Fr = V / sqrt(g * D_h), where D_h = A / T (hydraulic depth [m]).*
*y_2 / y_1 = 0.5 * (sqrt(1 + 8 * Fr_1^2) - 1)*
*Delta_E = (y_2 - y_1)^3 / (4 * y_1 * y_2) [Energy dissipation head loss, m].*

Darcy-Weisbach Pipe Head Loss & Colebrook-White Friction Factor:

Equations (6-7): *h_f = f * (L / D) * (V^2 / (2 * g))*
*1 / sqrt(f) = -2.0 * log10((epsilon / (3.7 * D)) + (2.51 / (Re * sqrt(f))))*

Joukowsky Water Hammer Pressure Surge Wave Equation:

Equations (8-9): *Delta_P = rho * a * Delta_V, Delta_H = (a / g) * Delta_V*
Where rho = Fluid density [kg/m³], a = Acoustic pressure wave velocity [m/s], Delta_V = Rapid velocity reduction [m/s].

2.2 Academic Reference Table: Hydraulics & Pipe Flow

Function Identifier	Governing Equation / Formulation	Input Parameters & SI Units	Return
manning_discharge	Q = (1/n)*A*(R_h)^(2/3)*S_0^(1/2)	area [m²], hydraulic_radius [m], slope [m/m], roughness [s/m^(1/3)]	Q [m³/s]
manning_velocity	V = (1/n)*(R_h)^(2/3)*S_0^(1/2)	hydraulic_radius [m], slope [m/m], roughness [s/m^(1/3)]	V [m/s]
manning_n	n = (1/Q)*A*(R_h)^(2/3)*S_0^(1/2)	discharge [m³/s], area [m²], hydraulic_radius [m], slope [m/m]	n [-]
manning_s0	S_0 = (Q*n / (A*R_h^(2/3)))^2	discharge [m³/s], roughness [s/m^(1/3)], area [m²], hydraulic_radius [m]	S_0 [m/m]
chezy_velocity	V = C * sqrt(R_h * S_0)	chezy_c [m^(1/2)/s], hydraulic_radius [m], slope [m/m]	V [m/s]
strickler_velocity	V = k_st * R_h^(2/3) * S_0^(1/2)	strickler_k [m^(1/3)/s], hydraulic_radius [m], slope [m/m]	V [m/s]

froude_number	Fr = V / sqrt(g * hydraulic_depth)	velocity [m/s], hydraulic_depth [m], gravity [9.80665 m/s²]	Fr [-]
hydraulic_jump_sequent_depth	y2 = 0.5*y1*(sqrt(1+8*Fr1^2)-1)	y1 [m], froude1 [-]	y2 [m]
hydraulic_jump_energy_loss	EL = (y2 - y1)^3 / (4 * y1 * y2)	y1 [m], y2 [m]	EL [m]
critical_depth_rectangular	yc = (q^2 / g)^(1/3)	unit_discharge q [m²/s], gravity [9.80665 m/s²]	yc [m]
darcy_weisbach_head_loss	hf = f * (L/D) * (V^2 / (2*g))	friction_factor [-], length [m], diameter [m], velocity [m/s]	hf [m]
colebrook_white_friction	1/sqrt(f) = -2*log10(eps/(3.7D) + 2.51/(Re*sqrt(f)))	reynolds_number [-], roughness_height [m], diameter [m]	f [-]
swamee_jain_friction	f = 0.25 / [log10(eps/(3.7D) + 5.74/Re^0.9)]^2	reynolds_number [-], roughness_height [m], diameter [m]	f [-]
hazen_williams_head_loss	hf = 10.67 * L * Q^1.852 / (C^1.852 * D^4.87)	length [m], discharge [m³/s], c_factor [-], diameter [m]	hf [m]
water_hammer_pressure_surge	dP = rho * a * dV	density [kg/m³], wave_speed [m/s], velocity_change [m/s]	dP [Pa]

3. SURFACE WATER HYDROLOGY & RUNOFF (hydroaj.hydrology)

The hydrology module implements fundamental analytical and empirical relationships for catchment runoff, infiltration mechanics, time of concentration, and unit hydrograph synthesis.

3.1 Governing Equations & Analytical Formulations

The Rational Method Peak Runoff:

> **Equation (10):** *Q_p = 0.2778 * C * I * A (in SI Metric units)*
> *Where Q_p = Peak discharge [m³/s], C = Dimensionless runoff coefficient [-], I = Rainfall intensity [mm/hr], A = Catchment drainage area [km²] (or A in m² with Q = C * I * A / 3.6e6).*

NRCS / SCS Runoff Curve Number Method:

> **Equations (11-13):** *S = (25400 / CN) - 254 [Potential maximum retention, mm]*
> *I_a = lambda * S = 0.2 * S [Initial abstraction, mm]*
> *Q_d = (P - I_a)^2 / (P - I_a + S) for P > I_a, else 0 [Direct runoff depth, mm].*

Empirical Time of Concentration Formulations:

> **Equations (14-17):** *Kirpich: T_c = 0.0195 * L^(0.77) * S_0^(-0.385) [minutes; L in m, S_0 in m/m]*
> *California Culverts: T_c = 0.057 * (L^3 / H)^0.385 [minutes; L in m, H in m]*
> *FAA: T_c = (1.8 * (1.1 - C) * L^0.5) / S_percent^(0.333) [minutes; L in ft or m]*
> *SCS Time to Peak: T_p = 0.5 * D + t_L [hours; D = storm duration hr, t_L = lag time hr].*

Infiltration Models (Horton & Green-Ampt):

> **Equations (18-19):** *Horton: f(t) = f_c + (f_0 - f_c) * exp(-k * t) [Infiltration rate, mm/hr]*
> *Green-Ampt: f(t) = K * (1 + (psi * Delta_theta) / F(t)) [Infiltration capacity rate, mm/hr].*

3.2 Academic Reference Table: Hydrology & Runoff

Function Identifier	Governing Equation / Formulation	Input Parameters & SI Units	Return
rational_peak_flow	Q = 0.2778 * C * I * A	runoff_coefficient [-], intensity [mm/hr], area [m²]	Q [m³/s]
scs_runoff_depth	Qd = (P - 0.2*S)^2 / (P + 0.8*S)	precipitation [mm], curve_number [dimensionless]	Qd [mm]
scs_potential_retention	S = (25400 / CN) - 254	curve_number [dimensionless 0-100]	S [mm]
kirpich_tc	Tc = 0.0195 * L^0.77 * S0^(-0.385)	length [m], slope [m/m (decimal)]	Tc [min]
california_culverts_tc	Tc = 0.057 * (L^3 / H)^0.385	length [m], head_drop [m]	Tc [min]
scs_time_to_peak	Tp = 0.5 * D + tL	rainfall_duration [hours], lag_time [hours]	Tp [hours]
horton_rate	f(t) = fc + (f0 - fc)*exp(-k*t)	time [hours], initial_capacity	f [mm/hr]

		[mm/hr], steady_capacity [mm/hr], decay_constant [1/hr]	
horton_cumulative	F(t) = fc*t + ((f0 - fc)/k)*(1 - exp(-k*t))	time [hours], initial_capacity [mm/hr], steady_capacity [mm/hr], decay_constant [1/hr]	F [mm]
green_ampt_rate	f = K * (1 + (psi * dTheta) / F)	hydraulic_conductivity [m/s], suction_head [m], moisture_deficit [-], cumulative_depth [m]	f [m/s]
muskingum_coefficients	C0, C1, C2 from K and X	k [hours], x [dimensionless weighting factor 0-0.5], dt [hours]	Tuple (C0, C1, C2)
muskingum_routing_step	O2 = C0*I2 + C1*I1 + C2*O1	i1 [m³/s], i2 [m³/s], o1 [m³/s], c0 [-], c1 [-], c2 [-]	O2 [m³/s]

4. HYDRAULIC STRUCTURES & FLOW CONTROL (hydroaj.structures)

The structures module encompasses discharge rating equations for primary flow measurement structures, weirs, sluice gates, and energy dissipation basins.

4.1 Governing Equations & Analytical Formulations

Francis Sharp-Crested Rectangular Weir Equation:

> **Equation (20):** *Q = 1.84 * (L - 0.1 * n_e * H) * H^(3/2) (in SI Metric units)*
> *Where Q = Discharge [m³/s], L = Crest length [m], n_e = Number of end contractions (0, 1, or 2), H = Upstream head [m].*

Triangular V-Notch Weir Equation (Thomson Formula):

> **Equation (21):** *Q = (8 / 15) * C_d * sqrt(2 * g) * tan(theta / 2) * H^(5/2)*
> *For standard 90-degree notch (C_d approx 0.585): Q approx 1.38 * H^(2.5) [m³/s].*

Sluice Gate Discharge (Free vs Submerged):

> **Equations (22-23):** *Free Outflow: Q = C_d * B * b * sqrt(2 * g * y_1)*
> *Submerged Outflow: Q = C_d * B * b * sqrt(2 * g * (y_1 - y_3))*
> *Where B = Gate width [m], b = Gate vertical opening [m], y_1 = Upstream depth [m], y_3 = Tailwater depth [m].*

4.2 Academic Reference Table: Hydraulic Structures

Function Identifier	Governing Equation / Formulation	Input Parameters & SI Units	Return
francis_weir	Q = 1.84 * (L - 0.1*n_e*H) * H^(3/2)	length [m], head [m], end_contractions [integer 0, 1, 2]	Q [m³/s]
v_notch_weir	Q = (8/15)*Cd*sqrt(2g)*tan(theta/2)*H^(5/2)	head [m], notch_angle_deg [degrees (90, 60, 45, 30)], cd [-]	Q [m³/s]
v_notch_90deg	Q = 1.38 * H^(2.5)	head [m]	Q [m³/s]
broad_crested_weir	Q = 1.705 * C_d * L * H^(3/2)	length [m], head [m], discharge_coeff [-]	Q [m³/s]
sluice_gate_free	Q = Cd * B * b * sqrt(2*g*y1)	gate_opening [m], gate_width [m], head [m], cd [-]	Q [m³/s]
sluice_gate_submerged	Q = Cd * B * b * sqrt(2*g*(y1 - y3))	gate_opening [m], gate_width [m], upstream_head [m], tailwater_depth [m], cd [-]	Q [m³/s]
parshall_flume_free	Q = C * H_a^n	upstream_head [m], throat_width [m]	Q [m³/s]
usbr_stilling_basin_length	L_b = 4.3 * y2 (USBR Type III)	froude1 [-], y1 [m], y2 [m]	Lb [m]

5. GROUNDWATER MECHANICS & WELL FLOW (hydroaj.groundwater)

The groundwater module models saturated porous media transport, Darcy seepage flow, and well drawdown in confined and unconfined aquifers.

5.1 Governing Equations & Analytical Formulations

Darcy's Law for Seepage Flux and Volumetric Discharge:

Equations (24-26): *q = -K * (dh / dl) [Specific discharge / Darcy flux, m/s]*
v_s = q / n_e [Actual seepage velocity, m/s; n_e = effective porosity]
*Q = -K * A * (dh / dl) [Volumetric flow rate, m³/s].*

Theis Non-Equilibrium Well Drawdown Equation:

Equations (27-28): *s = (Q / (4 * pi * T)) * W(u)*
*u = (r^2 * S) / (4 * T * t)*
Where s = Drawdown [m], Q = Pumping rate [m³/s], T = Transmissivity [m²/s], S = Storativity [-], r = Radial distance [m], t = Elapsed time [s], W(u) = Exponential integral well function.

Cooper-Jacob Logarithmic Approximation (for u < 0.05):

Equation (29): *s = (2.303 * Q / (4 * pi * T)) * log10((2.25 * T * t) / (r^2 * S))*

Dupuit-Forchheimer Unconfined Steady-State Flow:

Equation (30): *q = 0.5 * K * (h_1^2 - h_2^2) / L [Discharge per unit width, m²/s].*

5.2 Academic Reference Table: Groundwater Mechanics

Function Identifier	Governing Equation / Formulation	Input Parameters & SI Units	Return
darcy_flow_1d	Q = -K * A * (dh / dl)	hydraulic_conductivity [m/s], area [m²], hydraulic_gradient [m/m]	Q [m³/s]
darcy_flux	q = -K * (dh / dl)	hydraulic_conductivity [m/s], hydraulic_gradient [m/m]	q [m/s]
seepage_velocity	vs = q / ne	darcy_flux [m/s], effective_porosity [-]	vs [m/s]
theis_drawdown	s = (Q / (4*pi*T)) * W(u)	pumping_rate [m³/s], transmissivity [m²/s], storativity [-], radius [m], time [seconds]	s [m]
theis_u_parameter	u = (r^2 * S) / (4 * T * t)	radius [m], storativity [-], transmissivity [m²/s], time [seconds]	u [-]
cooper_jacob_drawdown	s = (2.3*Q/(4*pi*T))*log10(2.25*T*t / (r^2*S))	pumping_rate [m³/s], transmissivity [m²/s], storativity [-], radius [m], time	s [m]

		[seconds]	
thiem_confined_well	Q = 2*pi*T*(h2 - h1) / ln(r2 / r1)	transmissivity [m²/s], head1 [m], head2 [m], radius1 [m], radius2 [m]	Q [m³/s]
dupuit_unconfined_flow	q = 0.5 * K * (h1^2 - h2^2) / L	hydraulic_conductivity [m/s], head1 [m], head2 [m], length [m]	q [m²/s]

6. SEDIMENT DYNAMICS & BOUNDARY SHEAR (hydroaj.sediment)

The sediment module addresses fluid boundary layer shear stress, incipient particle motion, suspended sediment concentration profiles, and bedload transport rates.

6.1 Governing Equations & Analytical Formulations

Boundary Bed Shear Stress and Shear Velocity:

> **Equations (31-32):** *tau_0 = gamma * R_h * S_0 = rho * g * R_h * S_0 [N/m² = Pa]*
> *u_* = sqrt(tau_0 / rho) = sqrt(g * R_h * S_0) [Shear velocity, m/s].*

Dimensionless Shields Parameter:

> **Equation (33):** *theta = tau_0 / ((gamma_s - gamma) * d_50) = tau_0 / ((rho_s - rho) * g * d_50)*
> *Incipient motion occurs when theta >= theta_c (typically 0.03 to 0.06 for turbulent boundary layer).*

Rouse Suspended Sediment Concentration Profile:

> **Equations (34-35):** *C(y) / C_a = (((h - y) / y) * (a / (h - a)))^Z_R*
> *Where Z_R = w_s / (kappa * u_*) [Rouse number; w_s = settling velocity m/s, kappa = von Karman constant 0.4].*

6.2 Academic Reference Table: Sediment Transport

Function Identifier	Governing Equation / Formulation	Input Parameters & SI Units	Return
boundary_shear_stress	tau_0 = gamma * R_h * S_0	unit_weight [N/m³], hydraulic_radius [m], slope [m/m]	tau_0 [N/m²]
shear_velocity	u* = sqrt(g * R_h * S_0)	hydraulic_radius [m], slope [m/m], gravity [9.80665 m/s²]	u* [m/s]
shields_parameter	theta = tau_0 / ((gamma_s - gamma)*d50)	shear_stress [N/m²], grain_diameter [m], sediment_density [kg/m³], fluid_density [kg/m³]	theta [-]
critical_shear_stress	tau_c = theta_c * (gamma_s - gamma) * d50	critical_shields [-], grain_diameter [m], sediment_density [kg/m³], fluid_density [kg/m³]	tau_c [N/m²]
rouse_number	Z_R = ws / (kappa * u*)	settling_velocity [m/s], shear_velocity [m/s], von_karman [0.4]	Z_R [-]
rouse_concentration	C(y) = Ca * [((h-y)/y)*(a/(h-a))]^ZR	reference_conc [mg/L], flow_depth [m], eval_height [m], ref_height [m],	C(y) [mg/L]

		rouse_number [-]	
meyer_peter_muller_bedload	MPM empirical bedload formulation	shear_stress [N/m²], grain_diameter [m], sediment_density [kg/m³], fluid_density [kg/m³]	q_b [kg/(m*s)]

7. QUALITY ASSURANCE, BENCHMARKS & VERIFICATION REPORT

To establish HydroAj as an industry and academic standard, the computational engine was subjected to a rigorous two-tier validation regimen: 1) Analytical accuracy verification against published textbook solutions, and 2) Throughput scalability profiling.

7.1 Automated Verification Suite Results

The test framework is executed under Python 3.11 with pytest across both unit test assertions and complete textbook workbook solutions:

Verification Test Suite	Scope of Coverage	Test Count	Execution Status
test_new_expert_formulas.py	Advanced hydrology, hydraulics & structures	27 Test Cases	100% PASSED
run_all_workbook_examples.py	Standard engineering workbook practice problems	112 Practice Cases	100% PASSED
Pytest Core Test Suite	Expansions, inversions, unit consistency	108 Unit Tests	100% PASSED
Package Integrity Check (twine check)	PyPI metadata, README, author credentials	Wheel + Tar.gz	100% PASSED

7.2 Scalability Benchmarks: 1,000,000 Element Execution

A benchmark evaluating 1,000,000 open-channel cross-sections was executed comparing pure Python loops, vectorized NumPy, and the HydroAj Rust SIMD engine:

Implementation Approach	Execution Time (1M rows)	Throughput (ops/sec)	Relative Speedup
Pure Python (for-loop)	1.842 seconds	542,800 rows/s	1.0x (Baseline)
Standard NumPy Vectorized	0.082 seconds	12,195,000 rows/s	22.5x
HydroAj Rust SIMD Engine	0.019 seconds	52,631,000 rows/s	97.0x Speedup

8. DISCLAIMER, QUALITY ASSURANCE & ERRATA REPORTING

OFFICIAL DISCLAIMER: *Quality Assurance & Community Peer-Review Notice:*

HydroAj has been engineered to the highest standard of mathematical and physical rigor. Every analytical routine, numerical solver, and inversion within this library has been validated against benchmark textbook solutions and official engineering standards.

However, due to the breadth of hydraulic and hydrological applications across diverse geotechnical and environmental conditions, numerical boundary cases or unexpected floating-point precision behaviors may arise. If any calculation discrepancy, numerical inconsistency, or potential software defect is identified, engineers and researchers are requested to report it immediately via the official GitHub Issue Tracker: https://github.com/ajaybhattarai-123/hydroaj/issues.

Your direct feedback facilitates rapid verification, continuous refinement, and the perpetuation of an error-free, open-source standard for the global civil engineering community.

9. ACADEMIC CITATION (BibTeX)

Researchers, academic faculty, and practicing engineers utilizing HydroAj in academic research, published papers, master's/doctoral theses, or formal project design briefs are kindly requested to cite the framework as follows:

```
@software{bhattarai2026hydroaj,
  author       = {Bhattarai, Ajay},
  title        = {{HydroAj: High-Performance Water Resources Engineering
Computational Framework}},
  year         = {2026},
  publisher    = {GitHub},
  version      = {0.1.1},
  url          = {https://github.com/ajaybhattarai-123/hydroaj},
  note         = {Open-source hybrid Rust-Python library. Available on PyPI:
https://pypi.org/project/hydroaj/}
}
```